



LangQuest

Benutzerhandbuch



LangQuest

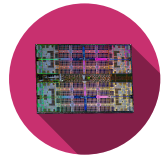
Computer Architecture

Information and Communication Technology •

Computer Architecture • Silvan Zahno

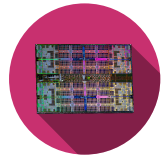
Silvan Zahno, Axel Amand

09.09.2026 • v1.0.0 • final



Inhalt

1 Einführung	3
2 Installation	4
2.1 Rust-Toolchain	4
2.1.1 Installation	4
2.1.2 cargo -Erweiterungen installieren	5
2.1.3 Überprüfung	6
2.2 LangQuest	7
2.2.1 Installation	7
2.2.2 Schnelleinstieg	8
2.2.3 Dateitypen dem Editor zuordnen	8
2.3 Zed-Code-Editor	8
2.3.1 Installation	8
2.3.2 Überprüfung	9
2.4 Just - Kommando-Runner	9
2.4.1 Installation	9
2.4.2 Überprüfung	10
2.5 GH CLI	10
2.5.1 Verification	11
3 LangQuest Anleitung	12
3.1 LangQuest starten	12
3.2 Übersichtsseite	13
3.2.1 Kurzbefehle im Überblick	14
3.3 Übungsseite	15
3.3.1 Theorie	15
3.3.2 Aufgaben	15
3.3.3 Debug	15
3.3.4 Ausgabe	16
3.3.5 Tipps und Lösung	16
3.3.6 Kurzbefehle im Überblick	17
3.4 Fehlerbehebung	18
Glossary	20



1 | Einführung

LangQuest wie auf der GitHub-Seite beschrieben:

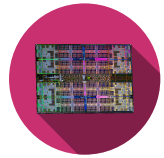


Ein interaktives, terminalbasiertes Tool zum Ausführen von Programmierübungen. Inspiriert von Rustlings und 100 Exercises to Learn Rust erweitert **LangQuest (lq)** das Konzept auf mehrere Sprachen - arbeiten Sie an praktischen Übungen in Rust, Go, C++, Python, **RISC-V Instruction Set Architecture (RISC-V)**-Assembly und Markdown mit Echtzeit-Feedback, Fortschrittsverfolgung und einem integrierten Hinweis-System.

Dieses Tool wurde von **Zahno Silvan** entwickelt und unter der **MIT License (MIT)** veröffentlicht. Der Quellcode ist unter <https://github.com/tschinz/langquest> verfügbar.

LangQuest ist ein interaktives Tool, dessen Schnittstelle dank **Ratatui** in der Befehlszeile ausgeführt wird und es Ihnen ermöglicht, praktische Übungen in Rust, Go, C++, Python, **RISC-V**-Assembly und Markdown mit Echtzeit-Feedback, Fortschrittsverfolgung und einem integrierten Hinweis-System zu bearbeiten.

Das folgende Dokument bietet eine umfassende Benutzeranleitung für LangQuest und behandelt Installation, Nutzung und Fehlerbehebung, damit Sie dieses interaktive Lernwerkzeug optimal nutzen können.



2 | Installation

Dieser Abschnitt behandelt die Installation aller Werkzeuge, die mit LangQuest verbunden sind. Arbeiten Sie die Unterabschnitte in Reihenfolge durch - einige Tools bauen auf vorherigen Installationen auf.

2.1 Rust-Toolchain



Figure 1 - Rust Programming Language

Verschiedene Werkzeuge werden über **Cargo**, den Paketmanager von Rust, installiert.

Rust wird über **rustup**, den offiziellen Installer der Rust-Toolchain, installiert und verwaltet. **rustup** installiert den Compiler (**rustc**), das Build- und Paketwerkzeug (**cargo**) sowie zusätzliche Ziele und Komponenten.

2.1.1 Installation

Vollständige Anweisungen finden Sie unter rust-lang.org/tools/install.

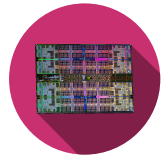
Öffnen Sie ein Terminal und führen Sie aus:

```
1 curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
```



Folgen Sie den Anweisungen am Bildschirm (die Standardinstallation wird empfohlen). Öffnen Sie danach das Terminal neu oder laden Sie die Umgebung:

```
1 source "$HOME/.cargo/env"
```



Laden Sie **rustup-init.exe** von <https://win.rustup.rs/> herunter und starten Sie es.

Falls **Microsoft Visual C++ (MSVC)**-Tools nicht installiert sind, bestätigen Sie die Installation auf Nachfrage:

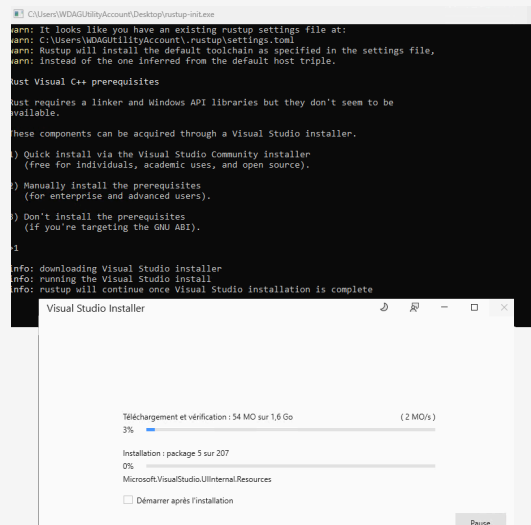


Figure 2 - **MSVC** installation through rustup

Fahren Sie dann mit der Installation fort (die Standardinstallation wird empfohlen):

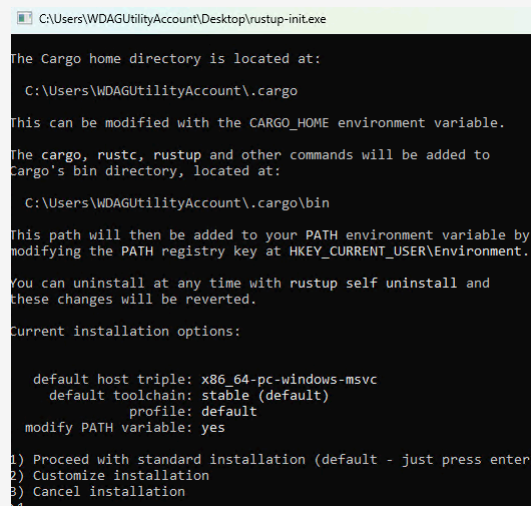
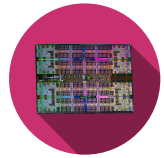


Figure 3 - Rust installation through rustup on Windows

2.1.2 cargo-Erweiterungen installieren

Führen Sie in einem neuen Terminal die folgenden Befehle aus, um nützliche **cargo**-Erweiterungen für Formatierung und Linting zu installieren:

```
1 # Install rustfmt for code formatting
2 rustup component add rustfmt
3 # Install clippy for linting and code analysis
4 rustup component add clippy
```



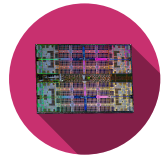
2.1.3 Überprüfung

Prüfen Sie, dass die folgenden Befehle eine Versionsnummer ohne Fehler ausgeben:



```
1 rustup --version
2 rustc --version
3 cargo --version
4 cargo fmt --version
5 cargo clippy --version
```

Alle Befehle sollten eine Versionsnummer ausgeben. Falls nicht, stellen Sie sicher, dass **\$HOME/.cargo/bin** (macOS/Linux) bzw. **%USERPROFILE%\.cargo\bin** (Windows) im **PATH** enthalten ist.



2.2 LangQuest



Figure 4 - LangQuest - vocabulary quiz tool

LangQuest ist ein terminalbasiertes Vokabel-Quiz-Tool zum Üben von Programmierbegriffen und -konzepten. Es wird als Binärdatei unter [LangQuest on Github](#) verteilt.

2.2.1 Installation



```
1 curl -fsSL https://raw.githubusercontent.com/tschinz/langquest/refs/heads/main/scripts/install_latest_release.sh | sh
```

LangQuest wird unter **\$HOME/.local/bin** installiert.



```
1 irm https://raw.githubusercontent.com/tschinz/langquest/refs/heads/main/scripts/install_latest_release.ps1 | iex
```

LangQuest wird unter **%LOCALAPPDATA%\Programs\lq\bin\lq.exe** installiert.

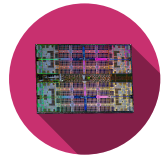
Prüfen Sie, dass der folgende Befehl eine Versionsnummer ohne Fehler ausgibt:

```
1 lq -k
```

Prüfen Sie, dass die Schlüssel den folgenden entsprechen:

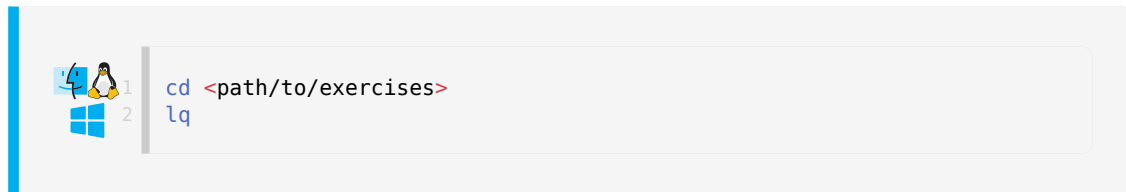


```
1 progress_key_sha256:
869e730b3d6be463c5474a41f802711943e330827a7c4ddd6bcd2dc9f4abc3a
2 attest_key_sha256:
af30b6a2512c11e8015da1522b55d61926f96591c3193a63f2d1b64f6f53b5b7
3 solution_key_sha256:
1d142c246028f02e928682c88b3280412d4ff82b0cbc275db68a236947c89283
```



2.2.2 Schnelleinstieg

Zum Start wechseln Sie in einen Ordner mit LangQuest-Übungen und führen aus:



Das Programm erkennt Struktur und Inhalte der Übungen automatisch.

Wenn der Befehl zum ersten Mal in einem Ordner ausgeführt wird, erstellt er mehrere Konfigurationsdateien:

- **lq.toml** - Konfigurationsdatei für LangQuest
 - Sollte in die Versionskontrolle aufgenommen werden
- **.lq.progress** - Fortschrittsdatei zur Verfolgung Ihres Fortschritts (verschlüsselt, nicht bearbeiten)
 - Sollte in die Versionskontrolle aufgenommen werden
- **.lq.attest** - Attestationsdatei für die Offline-Nutzung (verschlüsselt, nicht bearbeiten)
 - Maschinenspezifisch, **nicht in die Versionskontrolle aufnehmen**

2.2.3 Dateitypen dem Editor zuordnen

Sie können den Editor definieren, den LangQuest zum Öffnen von Übungsdateien verwendet. Diese Konfiguration befindet sich in **lq.toml**. Standardmässig ist es so konfiguriert, dass **zed** (<https://zed.dev/>) verwendet wird. Sie können es auf Ihren bevorzugten Editor ändern, z. B. **code** für VSCode oder **subl** für Sublime Text.

```
1 [ide]
2 bin = "/usr/local/bin/zed"
3 cmd = "<ide> <file>"
```

2.3 Zed-Code-Editor

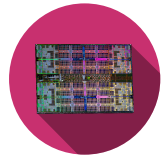


Figure 5 - Zed - high-performance code editor

Zed ist ein leistungsstarker, kollaborativer Code-Editor auf Rust-Basis. Er ist der primäre Editor in diesem Guide.

2.3.1 Installation

Besuchen Sie zed.dev und laden Sie den Installer für Ihre Plattform herunter.



Laden Sie die **.dmg** herunter, öffnen Sie sie und ziehen Sie *Zed* in den Ordner *Applications*.

Installieren Sie das Zed-CLI in Zed: Drücken Sie **cmd-shift-p** und geben Sie **cli:install cli binary** ein



Öffnen Sie ein Terminal und führen Sie das offizielle Installationsskript aus:

```
1 curl https://zed.dev/install.sh | sh
```



Laden Sie den **.exe**-Installer herunter und führen Sie ihn aus.

Fügen Sie **zed.exe** zu Ihrem **PATH** hinzu, falls dies nicht automatisch geschieht.

2.3.2 Überprüfung



Öffnen Sie Zed und bestätigen Sie, dass der Startbildschirm erscheint. Melden Sie sich mit Ihrem GitHub-Konto an, um **Artificial Intelligence (AI)**-Funktionen zu aktivieren.

Führen Sie den folgenden Befehl in einem Terminal aus, um zu überprüfen, ob Zed installiert ist und in Ihrem **PATH** verfügbar ist:

```
1 zed --version
```

2.4 Just - Kommando-Runner

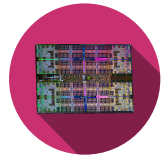
just ist ein praktischer Command-Runner (ähnlich wie **make**), der ein **justfile** im Projekt liest. Er wird als binäre Rust-Crate verteilt und über **cargo** installiert.

2.4.1 Installation



```
1 cargo install just
```

Die Erstinstallation kann einige Minuten dauern, da Crate und Abhängigkeiten heruntergeladen und kompiliert werden.



2.4.2 Überprüfung

Prüfen Sie, dass der folgende Befehl eine Versionsnummer ohne Fehler ausgibt.



```
1 just --version
```

2.5 GH CLI

GitHub CLI ist ein Kommandozeilenwerkzeug zur Interaktion mit GitHub-Repositories und -Diensten. Es wird verwendet, um Pull Requests, Issues und andere GitHub-Funktionen direkt vom Terminal aus zu verwalten.

Es wird von LangQuest verwendet, um die Dateien basierend auf dem registrierten Github-Benutzer zu identifizieren.

Verwenden Sie brew zur Installation:



```
1 brew install gh
```

Installieren Sie es über Ihren Paketmanager, falls verfügbar, z. B.: Debian/Ubuntu



```
1 sudo apt install gh # Ubuntu
2 sudo dnf install gh # Fedora
```

Entweder verwenden Sie WinGet, wenn es auf Ihrem System installiert ist:

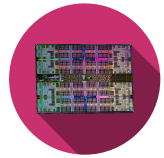


```
1 winget install --id GitHub.cli
```

Oder laden Sie den Installer für Ihr System direkt von cli.github.com herunter und führen Sie ihn aus.

Führen Sie dann den folgenden Befehl einmal aus, um sich mit Ihrem GitHub-Konto zu authentifizieren. Folgen Sie den Anweisungen, um den Authentifizierungsprozess abzuschließen, indem Sie sich über einen Webbrowser anmelden oder ein persönliches Zugriffstoken verwenden:

```
1 gh auth login
```



```
PS F:\> gh auth login
? Where do you use GitHub? GitHub.com
? What is your preferred protocol for Git operations on this host? HTTPS
? Authenticate Git with your GitHub credentials? Yes
? How would you like to authenticate GitHub CLI? [Use arrows to move, type to filter]
> Login with a web browser
  Paste an authentication token
```

Figure 6 - GitHub CLI-Installation

2.5.1 Verification

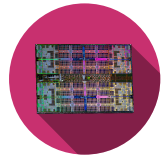
Führen Sie

```
1 gh auth status
```



aus, um sicherzustellen, dass das Konto korrekt verknüpft ist:

```
PS F:\dev_work\langquest> gh auth status
github.com
✓ Logged in to github.com account Qamma (keyring)
- Active account: true
- Git operations protocol: https
- Token: gho_*****
- Token scopes: 'gist', 'read:org', 'repo', 'workflow'
```



3 | LangQuest Anleitung

3.1 LangQuest starten

LangQuest verlangt keinen bestimmten Speicherort für Übungen, aber die Verzeichnisstruktur muss einem festen Layout folgen:

```
my-exercises/
├─ lq.toml                <-- auto-created by lq on first run
├─ 01-basics/             <-- module (prefixed with NN-)
│   └─ 01-hello-world/    <-- exercise (prefixed with NN-)
│       ├── 01-theory.md
│       ├── 02-task.md
│       ├── main.rs
│       └─ solution/
│           ├── main.rs
│           └─ solution.md
├─ 02-variables/
│   └─ ...
└─ 02-control-flow/
    └─ ...
```

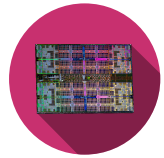
Speichern Sie die Übungen Ihres Kurses in einem eigenen Ordner.

Wechseln Sie im Terminal in den Ordner mit den Übungen und starten Sie LangQuest mit **lq**:

```
cd <path/to/exercises>
lq
```



Es wird empfohlen, LangQuest in einem in **Zed** geöffneten Terminal zu starten, damit die Übungsdateien direkt im selben Editor geöffnet werden. Die Screenshots in diesem Leitfaden stammen aus Zed.



3.2 Übersichtsseite

Öffnen Sie den Ordner mit den Übungen in **Zed**:

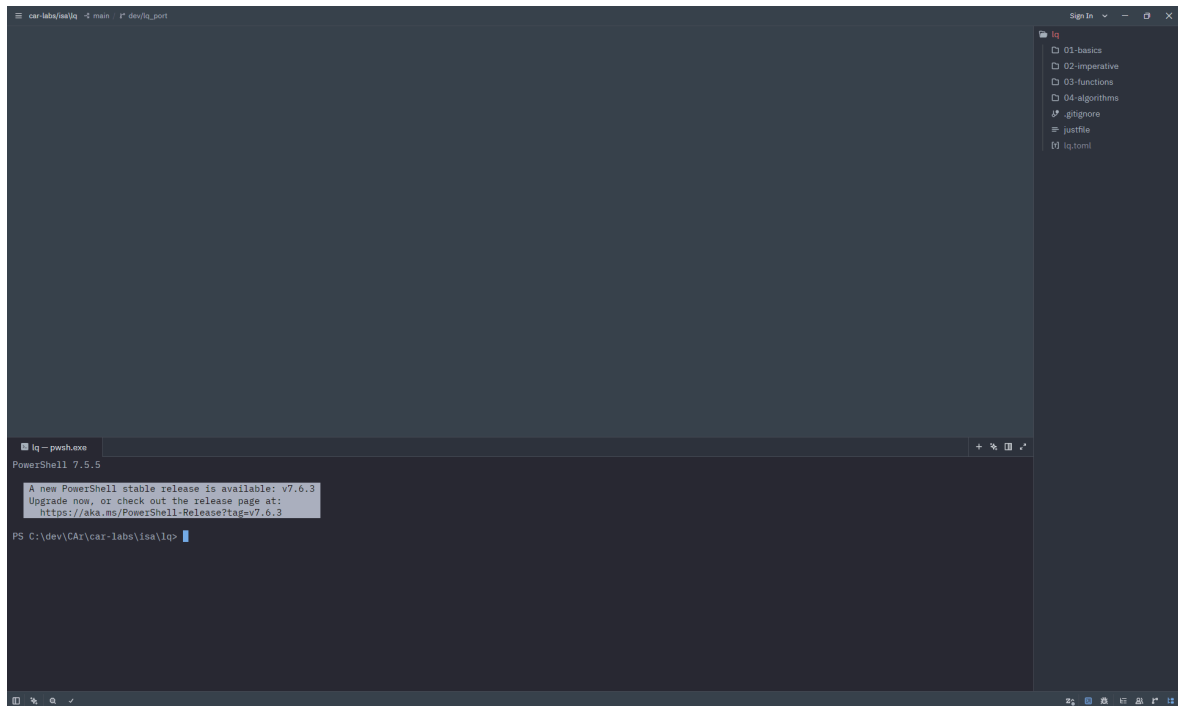


Figure 7 - Open exercises in Zed

Öffnen Sie das Terminal über **View** ⇒ **Terminal Panel**, geben Sie **lq** ein und drücken Sie **Enter**:

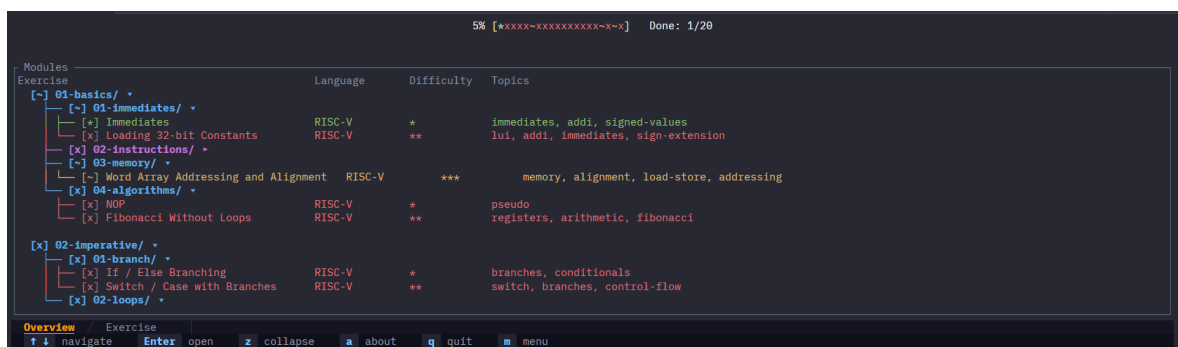
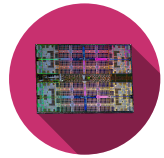


Figure 8 - **lq** in Zed



Wenn das Programm nicht startet, lesen Sie die Terminal-Ausgabe. Die meisten Fehler entstehen, wenn **lq** ausserhalb eines Übungsordners gestartet wird oder das Layout ungültig ist.



Die Programmoberfläche sieht wie folgt aus:

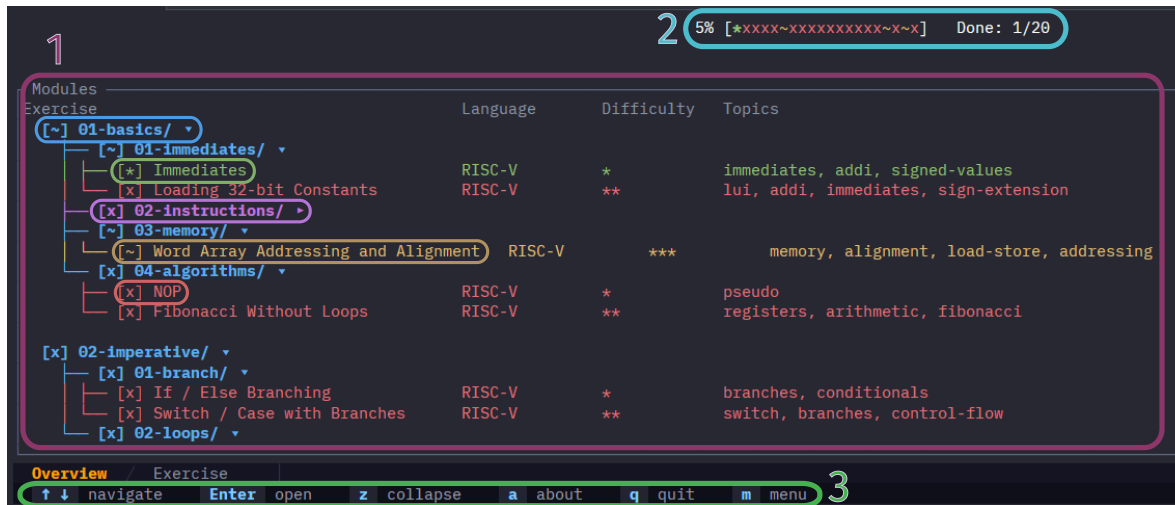


Figure 9 - **lq** overview page

1. Übungsliste

Gefundene Übungen werden geordnet angezeigt. Navigieren Sie mit **↓ ↑** und wählen Sie mit **Enter**.

- **Blaue** Zeilen markieren Module/Gruppen von Übungen. Mit ***z*** lassen sich Module ein- und ausklappen.
- Die **violette** Zeile zeigt die Cursor-Position.
- Die **grüne** Zeile markiert vollständig abgeschlossene Übungen.
- Die **gelbe** Zeile markiert teilweise abgeschlossene Übungen.
- Die **rote** Zeile markiert noch nicht abgeschlossene Übungen.

Für jede Übung zeigt die Ansicht Sprache (Rust, Go, C++, Python, RISC-V-Assembler und Markdown), relativen Schwierigkeitsgrad innerhalb des Moduls sowie behandelte Themen.

3.2.1 Kurzbefehle im Überblick

- **↓ ↑:** in der Übungsliste navigieren
- **Enter:** ein Modul erweitern / eine Übung auswählen
- ***z*:** alle Module in der Liste ein-/ausklappen

2. Fortschritt

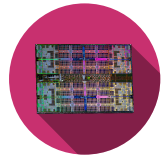
Die Fortschrittsleiste zeigt, wie viele Übungen im Kurs abgeschlossen sind. Sie wird pro Übung wie folgt aktualisiert:

- *: Übung vollständig abgeschlossen
- ~: Anforderungen teilweise erfüllt
- x: Anforderungen nicht erfüllt

3. Menu

Liste der für die aktuelle Ansicht verfügbaren Tastaturbefehle. Mit ***m*** blenden Sie das Menü ein oder aus.

- ***a***: die About-Seite anzeigen
- ***q*** / ***Esc***: Programm beenden
- ***m***: Menü mit Kurzbefehlen ein-/ausblenden



3.3 Übungsseite

Zwischen den Ansichten wechseln Sie mit $\leftarrow \rightarrow$. Theory $\Leftrightarrow$ Task $\Leftrightarrow$ Debug $\Leftrightarrow$ Output $\Leftrightarrow$ Solution (falls freigeschaltet).

3.3.1 Theorie

Beim Öffnen einer Übung wird der Theorie-Tab angezeigt. Er enthält eine kurze Beschreibung und Erinnerungen an die Kernkonzepte zur Lösung.

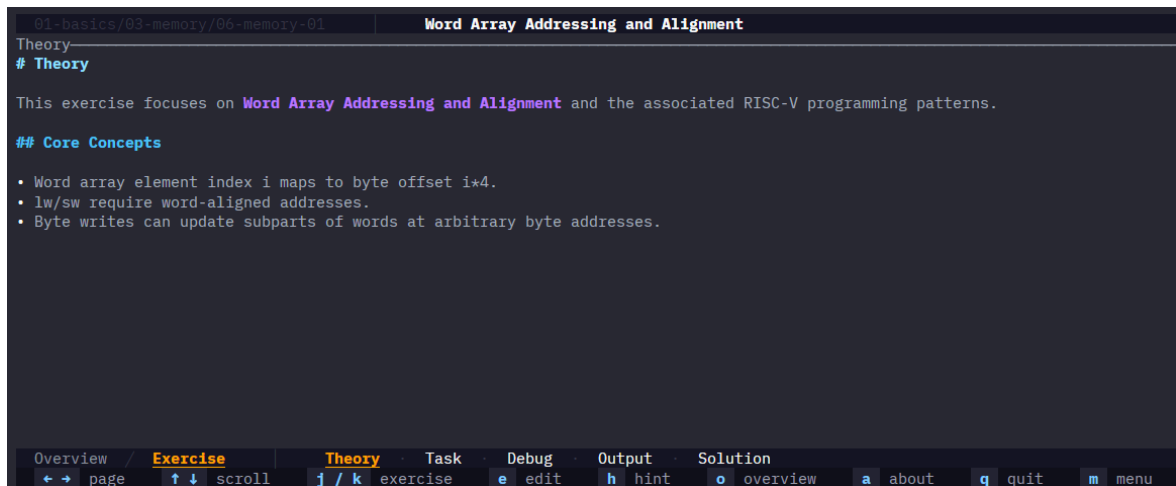


Figure 10 - Theory tab

3.3.2 Aufgaben

Die Aufgabenansicht enthält die Liste der zu erledigenden Aufgaben, Codeausschnitte, Hinweise ...

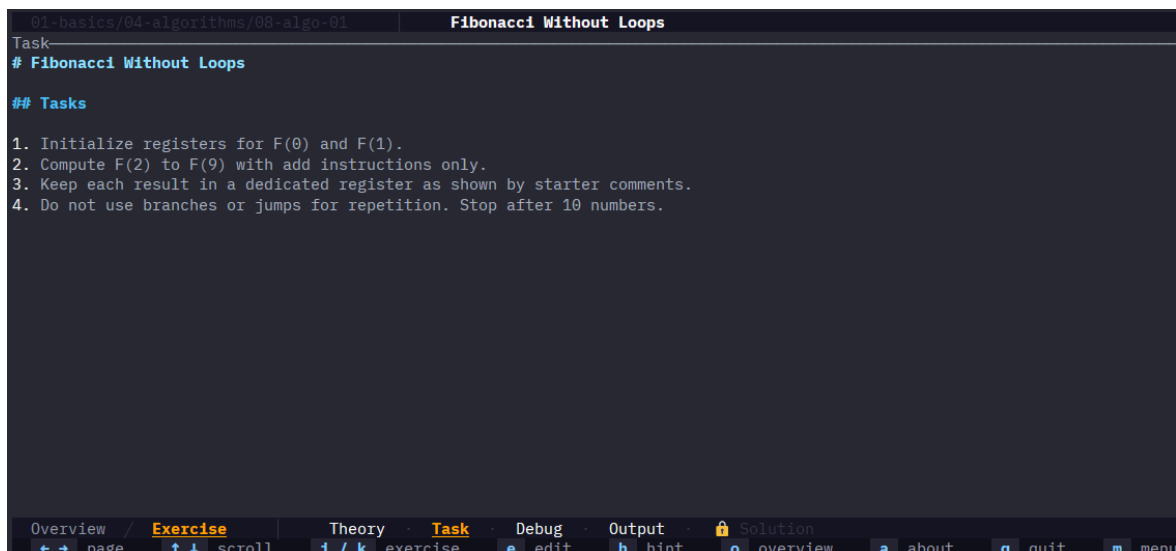


Figure 11 - Tasks tab

3.3.3 Debug

Das Debug-Fenster zeigt die Ausgaben von **stdout** und **stderr** des getesteten Programms. *Dieses Fenster ist nicht nützlich für RISC-V und Markdown.*

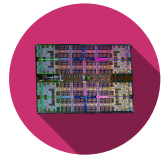


Figure 12 - Debug tab

3.3.4 Ausgabe

Das Ausgabefenster zeigt die Testergebnisse. Die Tests sind direkt im Übungscode definiert; schauen Sie dort bei Fehlern nach, um sie besser zu verstehen.

Es zeigt die Gesamtzahl bestandener Tests sowie die fehlgeschlagenen, im jeweils sprachspezifischen Format.

Der Schwellenwert rechts ist der Mindestprozentsatz bestandener Tests, damit die Übung als **teilweise abgeschlossen** oder **vollständig abgeschlossen** gilt.

Dieser Schwellenwert ist vor allem für Markdown-Übungen relevant, da sie anders getestet werden.

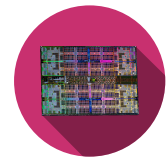
Figure 13 - Output tab

Drücken Sie ***e***, um die Übung im zuvor in Abschnitt 2.2.3 festgelegten Standardeditor zu öffnen. Ist derselbe Editor wie für **lq** gesetzt (z. B. Zed), öffnet sich die Datei in einem neuen Tab im selben Fenster.

3.3.5 Tipps und Lösung

Standardmässig ist die Lösung gesperrt, wie das Schloss-Symbol zeigt  **Solution**
Sie wird unter einer der beiden Bedingungen freigeschaltet:

- Der Abschlussschwellenwert der Übung ist erreicht.
- Alle Tipps sind freigeschaltet.



Zum Freischalten der Tipps drucken Sie ***h***; dann erscheint folgende Ansicht im Tab **Output**:

```
01-basics/04-algorithms/08-algo-01 | Fibonacci Without Loops
Output
Set the two seed values first: 0 and 1.
[HINT 2/2]
For each next term, add the previous two registers.

⚠ The solution will be unlocked. Are you really sure?
Press 'h' again to confirm, or any other key to cancel.

Overview / Exercise Theory Task Debug Output Solution
← → page ↑ ↓ scroll j / k exercise e edit h hint o overview a about q quit m menu
```

Figure 14 - Output tab

Sobald alle Tipps freigeschaltet sind, entsperrt ein erneutes Drücken von **"h"** die Lösung mit folgender Ansicht:

```
01-basics/02-instructions/01-instruction-01 | Arithmetic Expression (Positive Values)
Solution
— Reference Solution —
# EXPECT_REG: t0 1
# EXPECT_REG: t1 2
# EXPECT_REG: t2 5
# EXPECT_REG: s0 -2

_start:
# a = b + c - d;
# t0 = b, t1 = c, t2 = d. s0 = d

# b = 1, c = 2, d = 5
addi t0, zero, 1
addi t1, zero, 2
addi t2, zero, 5
# t = b + c
add t3, t0, t1
# a = t - c
sub s0, t3, t2

Explanation
## Explanation

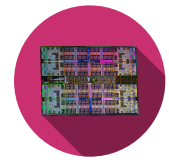
You practice expression lowering into primitive arithmetic instructions with explicit temporaries.
Read solution/main.asm together with the hints, then compare with main.asm to understand each implementation choice.

Overview / Exercise Theory Task Debug Output Solution
← → page ↑ ↓ scroll j / k exercise e edit h hint o overview a about q quit m menu
```

Figure 15 - Solution tab

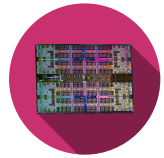
3.3.6 Kurzbefehle im Überblick

- **← →**: zwischen den Ansichten der Übung wechseln
- **↑ ↓**: in den Ansichten scrollen
- ***j*** / ***k***: zur vorherigen/nachsten Übung wechseln
- ***e***: Übung im Standardeditor öffnen
- ***h***: Tipps anzeigen + Lösung freischalten
- ***o***: zur Übersichtsseite zurück - Abschnitt 3.2
- ***a***: About-Seite anzeigen
- ***q*** / ***Esc***: Programm beenden
- ***m***: Kurzbefehls-Menu ein-/ausblenden

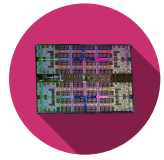


3.4 Fehlerbehebung

Problem	Ursache	Lösungsschritte
rustup , cargo oder rustc nicht verfügbar	Rust-Toolchain ist nicht korrekt installiert	1. Wiederholen Sie die Rust-Installation über https://rust-lang.org/tools/install/. 2. Starten Sie die Terminal-Sitzung neu. 3. Prüfen Sie mit rustup --version, rustc --version und cargo --version.
Befehl lq nicht gefunden	LangQuest ist nicht installiert oder Cargo-Binaries fehlen im PATH	1. Führen Sie cargo install --git https://github.com/tschinz/langquest aus. 2. Prüfen Sie cargo --version und lq --version. 3. Fügen Sie das Cargo-Bin-Verzeichnis zum PATH hinzu und starten Sie das Terminal neu.
Programm wird gestartet, beendet sich aber sofort	Falsches Arbeitsverzeichnis oder ungültige Übungsstruktur	1. Prüfen Sie die Terminal-Ausgabe. Meistens wird angezeigt, dass keine Übungen gefunden wurden. 2. Wechseln Sie in das Stammverzeichnis mit den Übungsmodulen. 3. Bestätigen Sie, dass die Ordnerstruktur dem erwarteten Muster NN-modul/NN-übung entspricht. 4. Starten Sie lq erneut von diesem Stammverzeichnis aus. 5. Eröffnen Sie ein Issue unter https://github.com/tschinz/langquest/issues.
Taste e öffnet Dateien nicht im erwarteten Editor	Dateityp-Zuordnungen fehlen oder zeigen auf eine andere Anwendung	1. Konfigurieren Sie Standard-Apps für .rs, .py, .go, .cpp, .md usw. neu. 2. Schließen und öffnen Sie den Editor erneut. 3. Starten Sie lq neu.
Lösung bleibt gesperrt	Abschlusschwelle nicht erreicht oder Tipps nicht vollständig freigeschaltet	1. Fixieren Sie Ihren Code, um die Tests zu bestehen, bis die Schwelle erreicht ist. 2. Drücken Sie h, bis alle Tipps freigeschaltet sind. 3. Drücken Sie h erneut, um den Lösungstab freizuschalten.
Programmoberfläche hängt oder reagiert nicht mehr	Derzeit arbeitet lq nur mit einem Thread. Wenn eine Übung geöffnet oder die Übungsdatei gespeichert wird, erfolgt eine Kompilierung + Ausführung der Da-	1. Prüfen Sie, ob der Übungscode keine Endlosschleife enthält. 2. Versuchen Sie eine Kompilierung außerhalb von lq. 3. Prüfen Sie, ob eine neuere Version von LangQuest verfügbar ist.



Problem	Ursache	Lösungsschritte
	tei. Wenn eine dieser Aktionen lange dauert, kann die Oberfläche hängen.	



Glossary

RISC-V – RISC-V Instruction Set Architecture: An open-source instruction set architecture for computer processors. [3](#), [3](#)

MIT – MIT License: A permissive free software license originating at the Massachusetts Institute of Technology (MIT). [3](#)

AI – Artificial Intelligence: Field of computer science focused on creating systems capable of performing tasks that typically require human intelligence. [9](#)

lq – LangQuest: An interactive programming exercise runner that supports multiple languages and provides real-time feedback. [3](#)

MSVC – Microsoft Visual C++: A compiler and development environment for C and C++ programming languages from Microsoft. [5](#), [5](#)